

BBM 418&AIN 433 – Programming Assignment 2: Object Tracking

Due date: Thursday, 28-5-2026, 11:59 PM.

Overview

In this assignment, you will implement a single-object tracking pipeline on video sequences. You will work with a single-object tracking dataset, where each sequence consists of image frames and ground-truth bounding boxes for a single target object.

Object tracking is the task of estimating the position of a target object in each frame of a video. In single-object tracking, the target object is given in the first frame, usually by a bounding box. The tracker is then expected to follow the same object in the remaining frames. This is different from object detection, where the goal is usually to detect all objects belonging to a certain category in an image. In this assignment, your goal is not to detect all objects in the scene, but to track the specific target object initialized in the first frame.

You will first load and visualize tracking sequences from the dataset. Then, you will generate local search-region samples from consecutive frames. These search regions will be used to train a CNN-based bounding-box regression model. Finally, you will use your trained model to track the target object in validation and test sequences, evaluate your results, and analyze the strengths and weaknesses of your tracker.

The main idea of the assignment is to learn how object localization and temporal information can be combined for tracking. You will observe how the tracker behaves under different challenges such as motion blur, fast motion, scale change, background clutter, object deformation, occlusion, and tracking drift.

Dataset

You will use a video sequence dataset. Each sequence contains an `img/` folder with video frames. Training and validation sequences include full ground-truth annotations. Test sequences include only the first-frame initialization box.

The dataset will be provided in the following structure:

```
Dog1/  
  img/  
    0001.jpg  
    0002.jpg  
    0003.jpg  
    ...  
  groundtruth_rect.txt
```

The `img` folder contains the frames of the video sequence. The `"groundtruth_rect.txt"` file contains the ground-truth bounding box of the target object for each frame. Each line of the annotation file corresponds to one frame.

The format of each line is:

`x,y,width,height`

For example, the following annotation file:

```
198,214,34,81  
197,214,34,81  
195,214,34,81
```

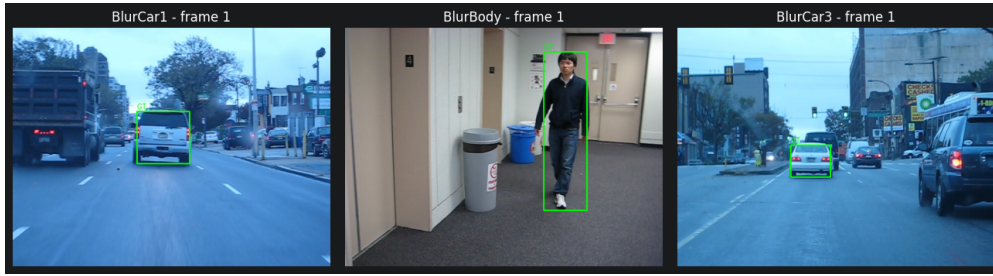


Figure 1: An example from the dataset. For each frame, a bounding box is drawn.

means that the target object is located at $(x=198, y=214)$ with width 34 and height 81 in the first frame. In the second frame, the target object is located at $(x=197, y=214)$ with width 34 and height 81, and so on.

The number of image frames and the number of bounding boxes in `"groundtruth_rect.txt"` must be the same. You can use this condition as a sanity check to validate there are no inconsistency in the data.

For test sequences, `"groundtruth_rect.txt"` is not provided. Instead, each test sequence contains `"init_rect.txt"`, which contains only the first-frame bounding box with the same (x, y, w, h) format. The first-frame box is used to initialize the tracker. The bounding boxes for the remaining test frames must be predicted by your tracker.

The dataset split will be provided separately. You will be given three files:

```
splits/train.txt
splits/val.txt
splits/test.txt
```

Each file will contain one sequence name per line. You must use only the provided split. Training sequences are used to train your model. Validation sequences are used to tune your implementation choices and hyperparameters. And also you will evaluate your model's performance at Part 5. Test sequences are used only for final evaluation.

We recommend that you use Colab (<https://colab.research.google.com/>) for the assignment, as all the assignment notebooks have been tested on Colab.

You will find the Jupyter Notebook file named `student_draft.ipynb`. To setup the Colab environment, you will need to upload the notebook file using the upload tab at <https://colab.research.google.com/>.

PART 1. Dataset Loading and Visualization (10 pts)

In the first part of the assignment, you will load dataset sequences and visualize the target object annotations.

You should implement a function that reads a sequence folder. This function should return the sorted image paths and the ground-truth bounding boxes. The image paths must be sorted according to frame order. The annotations must be loaded from `"groundtruth_rect.txt"` and stored in (x, y, w, h) format.

You should also implement a function for drawing a bounding box on an image. This will be used throughout the assignment for visualizing ground truth boxes, model predictions, and tracking results.

You are expected to visualize at least five frames from at least two different sequences. The visualized frames should show the target object bounding box. You should include these visualizations in your notebook.

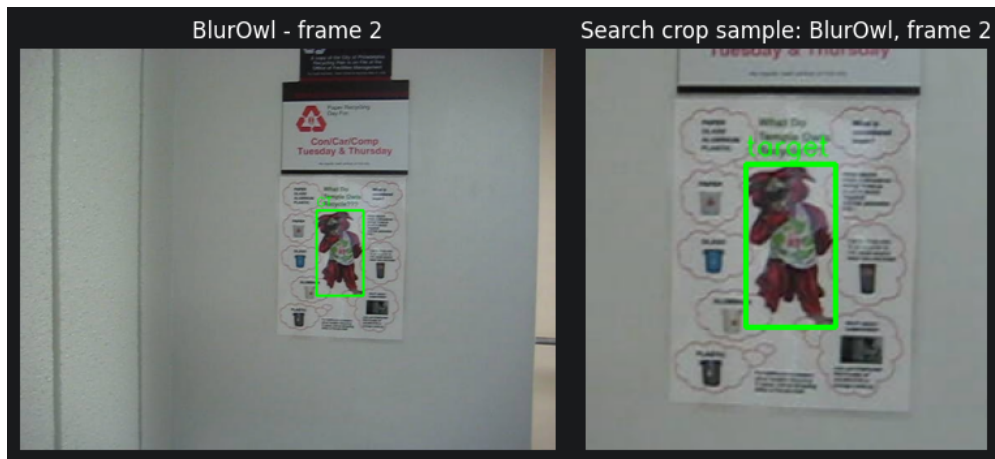


Figure 2: The original frame with bounding box, and the cropped version centered around the bounding box center

PART 2: Search-Region Sample Generation (15 pts)

In single-object tracking, the target object is initialized in the first frame. In later frames, the tracker usually searches around the previous target location instead of searching the whole image. This is because objects generally move smoothly between consecutive frames.

In this part, you will generate local search-region samples. These samples will later be used to train your CNN-based bounding-box regression model. For a pair of consecutive frames, assume that the target bounding box is known in frame $t - 1$ and frame t . The bounding box from frame $t - 1$ will be used to define a search region in frame t . The model will receive this search region as input and will learn to predict the target bounding box inside that crop.

For example, suppose we have:

Frame 1 bbox:	x=198, y=214, w=34, h=81
Frame 2 bbox:	x=197, y=214, w=34, h=81

To create a training sample for frame 2, you should use the frame 1 bounding box to crop a search region from frame 2. Then, you should convert the frame 2 bounding box into the coordinate system of this crop. Finally, the crop should be resized to 224×224 , and the target bounding box should be normalized to the $[0, 1]$ range.

The search region should be larger than the previous bounding box. For simplicity, use a square search region centered around the previous bounding box center. The size of the square search region can be defined as:

`crop_size = scale * max(previous_width, previous_height)`

where `scale` is a hyperparameter. A reasonable default value is 2.5. You can experiment with different values.

You should handle boundary cases carefully. Sometimes the search region may go outside the image boundaries. You may either clip the crop to image boundaries or pad the image before cropping. You should explain your choice in the notebook and make sure your coordinate conversions remain correct.

You should visualize some generated search-region samples. For each sample, show the search crop and draw the target bounding box inside the crop. This will help you verify that your coordinate transformation is correct.

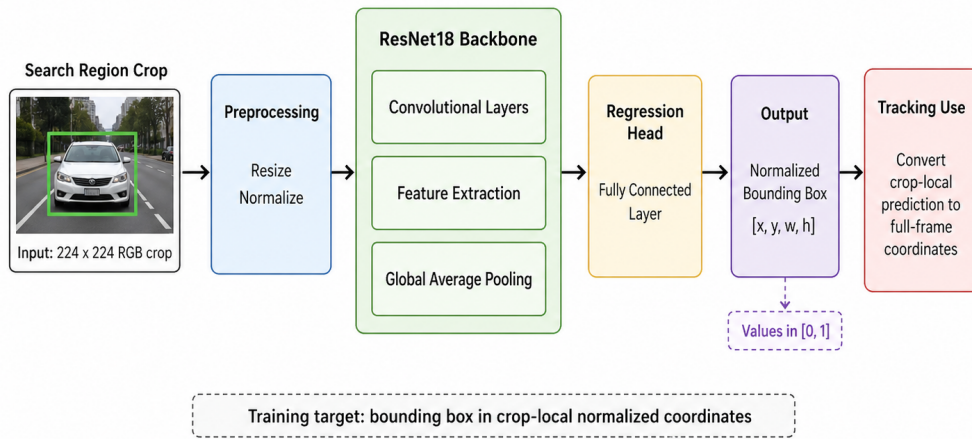


Figure 3: Overview of the CNN-based bounding-box regression model used in Part 3. A search-region crop resized to 224×224 is passed through a CNN backbone to extract features, and a regression head predicts the normalized bounding box (x, y, w, h) of the target object inside the crop.

PART 3: CNN-Based Bounding-Box Regression Model (25 pts)

In this part, you will train a CNN-based model that predicts the target object location inside a search region. The purpose of this model is not to classify the object category. Instead, the model learns to localize the target object in a local crop around its previous position.

The input to the model is a search-region crop resized to 224×224 . The output of the model is a four-dimensional bounding box representation:

`x, y, width, height`

These four values should represent the target bounding box inside the search crop, not inside the full video frame. The output coordinates must be normalized with respect to the crop size. Therefore, all output values should be in the range $[0, 1]$.

For example, if the model predicts $[0.40, 0.30, 0.20, 0.35]$, this means that the predicted box starts at 40% of the crop width and 30% of the crop height, and its width and height are 20% and 35% of the crop size, respectively. Since all crops are resized to 224×224 , this corresponds to a box inside the resized search crop. During tracking, this crop-local prediction must later be converted back to full-frame coordinates.

Model Architecture

You should use a convolutional neural network for bounding-box regression. A recommended approach is to use a pretrained ResNet18 model as the backbone. Since ResNet18 is originally designed for image classification, its final classification layer should be replaced with a regression layer that outputs four values.

The recommended model structure is as follows: (1) input search crop, (2) CNN feature extractor, (3) feature vector, (4) bounding-box regression layer, and (5) normalized bounding box (x, y, w, h) .

The model should output four values. You may use an output activation that restricts the predictions to the $[0, 1]$ range. This is useful because the target bounding boxes are also represented in normalized crop coordinates. However, if you use another strategy, you should make sure that the final predictions are valid normalized bounding boxes.

You may use pretrained ImageNet weights for the CNN backbone. This is recommended because the assignment dataset is relatively small, and training a CNN from scratch may not produce stable results. You can experiment with different architectures. But at minimum, you should implement the described model.

Image Preprocessing

All search crops should be resized to the same resolution before being given to the model. In this assignment, the expected input size is 224×224 RGB image. If you use a pretrained CNN backbone, you should normalize the input images using the same normalization scheme expected by that backbone. For ImageNet-pretrained ResNet models, this usually means using ImageNet mean and standard deviation values.

The same preprocessing steps must be used during training, validation, and tracking. A common mistake is to train the model with one preprocessing pipeline and then use a different preprocessing pipeline during tracking. This can significantly reduce performance.

Your preprocessing pipeline should include the following steps:

1. search-crop extraction
2. resizing to 224×224
3. conversion to a tensor
4. image normalization

You may include additional data augmentation during training, but do not apply random augmentation during validation or tracking.

Target Bounding-Box Representation

The training target must be the target object's bounding box inside the search crop. You should not train the model using full-frame bounding-box coordinates. For each training sample, you have the previous-frame bounding box, the current-frame image, and the current-frame bounding box. The previous-frame bounding box is used to define the search region in the current frame. Then, the current-frame bounding box is converted into the coordinate system of the search crop.

Conceptually, this conversion has two steps: (1) shift the current-frame bounding box according to the top-left corner of the search crop, and (2) normalize the shifted coordinates by the crop width and crop height. The target given to the model should therefore be the x-position inside the crop, the y-position inside the crop, the box width relative to the crop, and the box height relative to the crop. All four values should be normalized to the $[0, 1]$ range whenever possible.

This coordinate conversion is one of the most important parts of the assignment. If this step is incorrect, the model may appear to train correctly, but the tracker will produce incorrect predictions.

Training the model

Use a regression loss between the predicted bounding box and the ground-truth bounding box in crop-local normalized coordinates.

A suitable loss for this assignment is **Smooth L1 loss**. Smooth L1 loss is commonly used for bounding-box regression because it is less sensitive to large errors than mean squared error.

$$L(x) = \begin{cases} 0.5 * (x - y)^2 / \beta & , \text{if } |x - y| < \beta \\ |x - y| - 0.5\beta & , \text{otherwise} \end{cases}$$

Both the predicted bounding box and the target bounding box should have the same format and both should be normalized with respect to the search crop. You may experiment with other losses, but you should **explain** your choice if you do so.

You may use a standard optimizer such as **Adam**. The following hyperparameters are suggested as a reasonable starting point;

Batch size	Learning rate	# epochs	Optimizer	Loss function	Search-region scale
16 or 32	1×10^{-4}	5–15	Adam	Smooth L1 loss	2.5

You may and should change these values. However, you should report the final values you used in your notebook.

If training is too slow, you may reduce the number of epochs. If you encounter memory problems, reduce the batch size. You should still make sure that your model learns meaningful predictions and does not simply output almost constant boxes.

Your training procedure should include both training and validation stages. During training, the model should be updated using search-region samples from the training sequences. During validation, the model should be evaluated on search-region samples from the validation sequences without updating the model parameters.

At minimum, you should report, training loss per epoch, validation loss per epoch and validation crop-level mean IoU. You must also plot the training and validation loss curves. These curves help show whether the model is learning or overfitting.

The best model should be chosen after the training is completed and you should save the weights of the model to a file. Then it can be used for validation tracking and test prediction generation.

There are two different validation concepts in this assignment. The first is crop-level validation. This is used during model training. In crop-level validation, the model receives search crops generated from validation sequences, and its predictions are compared with the target bounding boxes inside those crops. This measures whether the CNN can localize the object inside a local search region.

The second is sequence-level tracking validation. This is used after the model has been trained. In sequence-level tracking validation, the tracker is initialized with the first-frame bounding box of a validation sequence. Then, it predicts the target box for each later frame. Each prediction is used to generate the next search region. This measures whether the full tracking pipeline works over time.

These two validation types are not the same. A model can perform reasonably well on crop-level validation but still fail during sequence-level tracking because tracking errors accumulate over time. You should report both.

Finally, you should also evaluate your model on the validation sequences by computing the mean IoU between predicted boxes and ground-truth boxes. In addition, you should visualize at least five validation predictions by drawing both the predicted box and the ground-truth box.

PART 4: Single-Object Tracking on Video Sequences (25 pts)

In this part, you will use your trained model to track the target object in validation and test sequences.

For each sequence, the tracker should start from the ground-truth bounding box in the first frame. This box is the initialization of the tracker. For the next frame, you should crop a search region around the previous predicted bounding box. Then, you should use your trained model to predict the target bounding box inside this search region. The predicted bounding box should be converted back to full-frame coordinates. This prediction will then be used as the previous bounding box for the next frame.

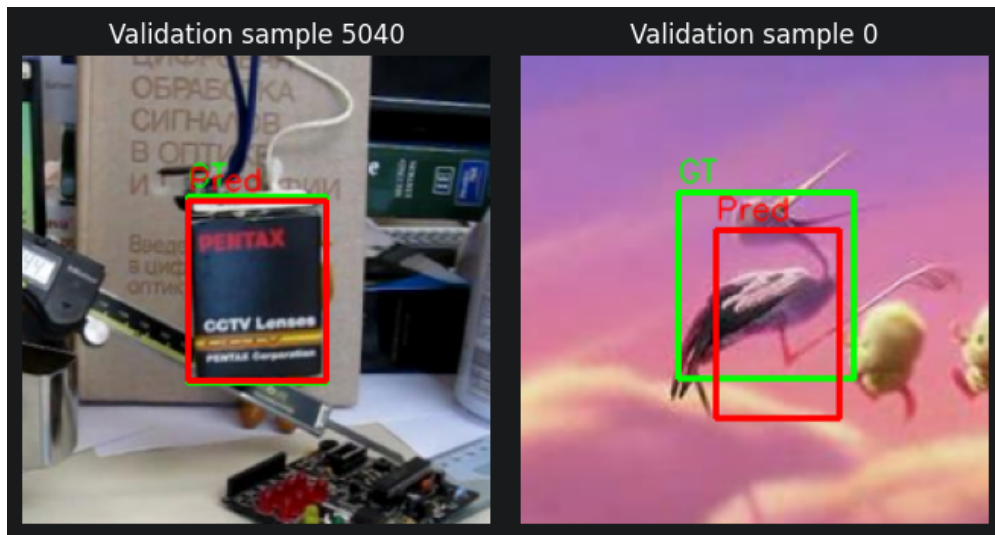


Figure 4: Two example predictions.

The tracking process should follow this structure:

1. Read first frame and first ground-truth bbox
2. Set this bbox as the initial prediction
3. For each next frame:
 - Crop search region around previous predicted bbox
 - Resize crop to 224×224
 - Run the trained model
 - Convert predicted crop-local bbox to full-frame bbox
 - Save prediction
 - Use this prediction for the next frame

The first prediction may be copied from the given first-frame ground-truth bounding box. All later predictions must be produced by your tracker.

For each test sequence, save your tracking predictions as a CSV file in the following format:

```
frame,x,y,w,h
1,198,214,34,81
2,197.2,214.1,34.5,80.7
3,195.4,213.9,34.8,81.2
```

The file should be saved as:

outputs/<sequence_name>_predictions.csv

The coordinates must be in **full-frame** image coordinates, not crop coordinates.

You should visualize the tracking results for at least three test sequences. For each selected sequence, show an early frame, a middle frame, and a late frame. Draw both the ground-truth bounding box and the predicted bounding box. Use different colors for ground truth and prediction.

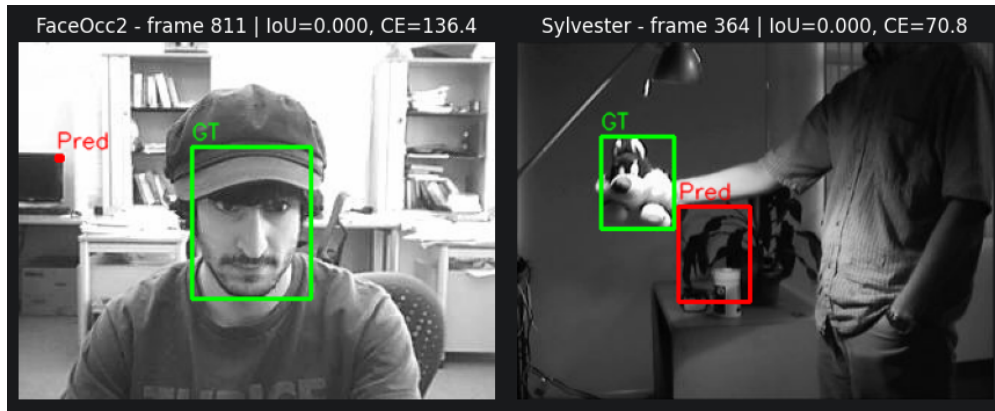


Figure 5: Two failed bounding box predictions.

PART 5: Evaluation and Analysis (25 pts)

In this part, you will evaluate your tracker using the ground-truth annotations.

You should compute the following metrics for each validation sequence. For the test sequences, you will only generate prediction CSV files.

- First, compute the Intersection over Union, or **IoU**, between the predicted bounding box and the ground-truth bounding box for each frame. The mean IoU over all frames will be used as one of the main evaluation metrics.
- Second, compute the success rate at $IoU \geq 0.5$. This metric measures the percentage of frames where the predicted bounding box overlaps sufficiently with the ground-truth bounding box.
- Third, compute the average center error. This is the average Euclidean distance between the center of the predicted bounding box and the center of the ground-truth bounding box.
- Finally, compute the failure count. In this assignment, a frame is considered a failure if $IoU < 0.1$

You should report your results in a table similar to the following:

Sequence	Mean IoU	Success Rate @0.5	Avg. Center Error	Failure Count
Sequence 1				
Sequence 2				
Sequence 3				
Average				

After reporting the numerical results, you should analyze your tracker. Your analysis should not only say whether the results are good or bad. You should explain why the tracker succeeds or fails in different cases. You should provide at least 3 success and fail cases with explanations.

Test Evaluation and Leaderboard (Bonus)

In addition to the validation evaluation reported in your notebook, your submitted test prediction files will be evaluated using hidden ground-truth annotations.

The test sequences contain only "init_rect.txt", which provides the first-frame bounding box for each sequence. You must use this initialization box to track the target object and generate

predictions for all frames. The full ground-truth annotations for the test sequences are hidden and will not be provided to students.

You are required to submit prediction CSV files for all test sequences. This part is not optional. However, the numerical performance on the test sequences will be used only for leaderboard-based bonus points. Your regular assignment grade will be based on the required implementation, validation evaluation, visualizations, discussion, and correctness of your submitted files.

For each test sequence, you must submit a CSV file in the following format:

`outputs/test/<sequence_name>_predictions.csv`

Each file must contain; (frame,x,y,w,h). The number of rows must be equal to the number of frames in the corresponding test sequence. Coordinates must be in full-frame pixel coordinates, not normalized crop coordinates.

The instructors will evaluate the submitted test predictions using the ground truth. The leaderboard score will be computed using the hidden test tracking metrics announced by the instructors, such as success rate at $\text{IoU} \geq 0.5$ and mean IoU. The exact leaderboard table may include additional metrics such as average center error and failure count.

Bonus points will be awarded according to the final leaderboard ranking:

Leaderboard Rank	Bonus Points
1st place	+20
2nd–5th place	+10
6th–10th place	+5

The leaderboard bonus can only be obtained if the submitted prediction files are valid and follow the required format. Invalid, missing, incomplete, or incorrectly formatted test prediction files will not be considered for leaderboard bonus.

Notes

You should pay attention to code readability, reproducibility, and organization. Your notebook should run from beginning to end without manual intervention.

Your implementation should satisfy the following requirements:

1. Use Python, and PyTorch for model training.
2. You may use OpenCV, NumPy, Pandas, Matplotlib, Torchvision, and other standard scientific Python libraries.
3. Respect the datasets original folder structure and use relative paths instead of absolute local paths.
4. Do not hard-code sequence-specific frame numbers or bounding boxes.
5. Organize your code into functions and classes, avoid repeated code, use clear variable names and comments.
6. Make sure your notebook can regenerate the output files.
7. You may use GPU acceleration if available. However, your code should also be able to run on CPU, although training may take longer.
8. You should use deterministic settings where possible. A cell that provides that given at the draft notebook. You should add more if you are introducing new randomness to the notebook.

What to Hand In

Your submitted solution should include the following:

- The filled-in Jupyter Notebook, as both your source code and report. The notebook should include (1) markdown cells reporting your written answers alongside any relevant figures and images and (2) well-commented code cells with reproducible results
- You should include a folder named **outputs** that contains prediction CSV files for **all** test sequences. Your notebook should be able to generate these output files in this directory.
- You should save the plots generated in the notebook inside the **outputs** folder.
- You should save the weights of the model used to generate the validation and test predictions inside a folder named **checkpoint**.
- **Important Note:** If the file size exceeds the limit allowed by the submission system, please upload your images to a cloud storage service (e.g., Google Drive, OneDrive, Dropbox, etc.) and include the shared link in your notebook file. Make sure the link remains accessible after the submission deadline.

You should prepare a ZIP file named name-surname(s)-pa2.zip containing the files stated above and, and submit it to `submit.cs.hacettepe.edu.tr`, where you will be assigned a submission.

Academic Integrity

All work on assignments must be done individually unless stated otherwise. You are encouraged to discuss with your other classmates about the given assignments, but these discussions should be carried out in an abstract way. That is, discussions related to a particular solution to a specific problem (either in actual code or in the pseudocode) will not be tolerated. In short, turning in someone else's work, in whole or in part, as your own will be considered as a violation of academic integrity. Please note that the former condition also holds for the material found on the web as everything on the web has been written by someone else.